

Coding!

Format and Plan for Today

- Today, we'll work through something called a jupyter notebook in pairs
- Each pair needs one computer; one of the pair members will type at a time—we'll signal to switch every 15 minutes
- Learning goals:
 - Declarations
 - Variables
 - Data types
 - Math operations
 - Loops
 - Conditionals



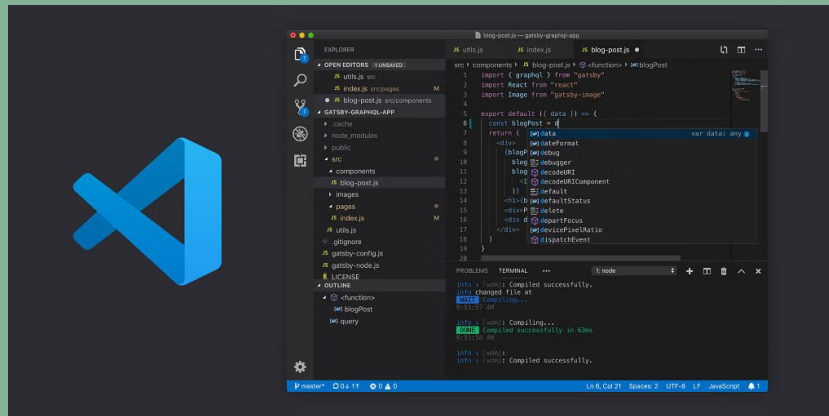
But first, let's get set up

jupyter

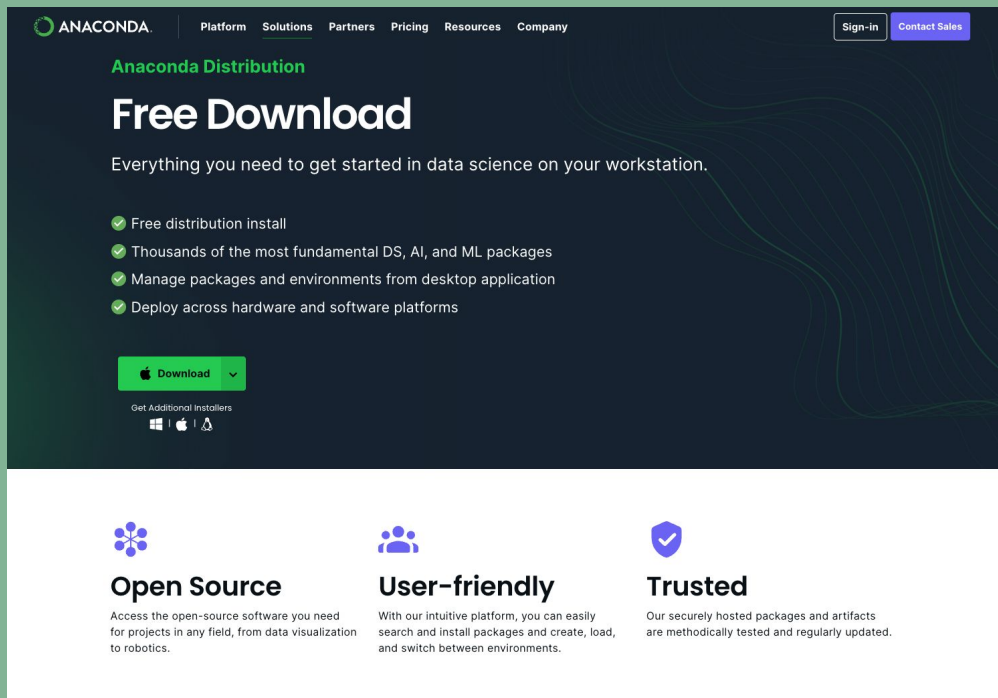
The screenshot displays the JupyterLab environment. The main window shows a notebook titled "In Depth: Linear Regression" with text explaining the importance of linear regression models. The left sidebar contains a file browser and a list of notebooks. The bottom panel shows a console with the output of a command. The top bar includes a launcher with icons for various environments like Python 3, C++11, and R. The bottom status bar indicates the current mode and the active notebook.

When coding in python, people often work in an IDE (integrated development environment), which is basically just a fancy text editor that has been optimized for building and running code in a user-friendly package – see below for some popular examples

VSCode



*In order to actually run python code, you also need to have python installed, and, for science tasks, this is usually most easily (and comprehensively) done using some variant of the **anaconda distribution***



The screenshot displays the Anaconda Distribution website. The top navigation bar includes links for Platform, Solutions, Partners, Pricing, Resources, and Company, along with Sign-in and Contact Sales buttons. The main heading is 'Anaconda Distribution Free Download', followed by the tagline 'Everything you need to get started in data science on your workstation.' Below this, a list of four benefits is shown, each preceded by a green checkmark: 'Free distribution install', 'Thousands of the most fundamental DS, AI, and ML packages', 'Manage packages and environments from desktop application', and 'Deploy across hardware and software platforms'. A prominent green 'Download' button with a dropdown arrow is present, with a link to 'Get Additional Installers' and icons for Windows, macOS, Linux, and Docker below it. The bottom section features three columns: 'Open Source' with a flower icon and text about open-source software; 'User-friendly' with a group of people icon and text about the intuitive platform; and 'Trusted' with a shield icon and text about secure hosting and updates.

ANACONDA. Platform Solutions Partners Pricing Resources Company Sign-in Contact Sales

Anaconda Distribution

Free Download

Everything you need to get started in data science on your workstation.

- ✓ Free distribution install
- ✓ Thousands of the most fundamental DS, AI, and ML packages
- ✓ Manage packages and environments from desktop application
- ✓ Deploy across hardware and software platforms

Download

Get Additional Installers

Windows macOS Linux Docker

Open Source

Access the open-source software you need for projects in any field, from data visualization to robotics.

User-friendly

With our intuitive platform, you can easily search and install packages and create, load, and switch between environments.

Trusted

Our securely hosted packages and artifacts are methodically tested and regularly updated.

Data comes in many shapes and sizes

TYPES	DESCRIPTION	EXAMPLES
Numeric	Represents numerical values	'int' , 'float' , 'complex'
Sequence	Represents a collection of ordered and indexed values	'list' , 'tuple' , 'range'
Text	Represents a sequence of characters	'str'
Mapping	Represents a collection of key-value pairs	'dict'
Boolean	Represents a binary truth value	'bool'

Variables are just like what you've seen in a math class, but fancier

In Python, we'll try to create a variable called "age" and assign a value to it, like this:

```
age = 15
```

Now, whenever you need to use that age value in your program, you can just use the variable name "age". For example:

```
print("I am " + str(age) + " years old.")
```



```
I am 15 years old.
```

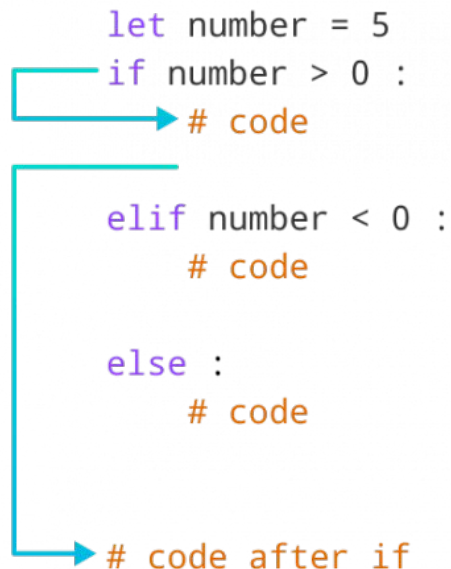
**We can also
do operations
on our
variables**

TYPES	DESCRIPTION	EXAMPLES
Arithmetic Operators	Used to perform arithmetic operations	'2 + 3' returns '5'
Comparison Operators	Used to compare values	'2 < 3' returns 'True'
Logical Operators	Used to combine and manipulate boolean values	'True and False' returns 'False'
Assignment Operators	Used to assign values to variables	'x=5'
Membership Operators	Used to check if a value is present in a sequence	'2 in [1, 2, 3]' return 'True'

Conditional Statements

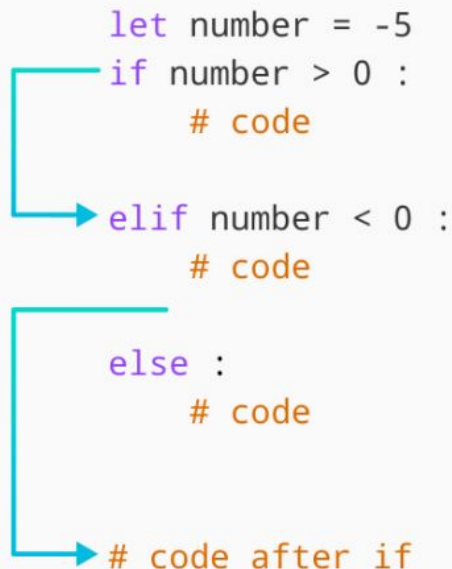
1st Condition is True

```
let number = 5
if number > 0 :
    # code
elif number < 0 :
    # code
else :
    # code
# code after if
```



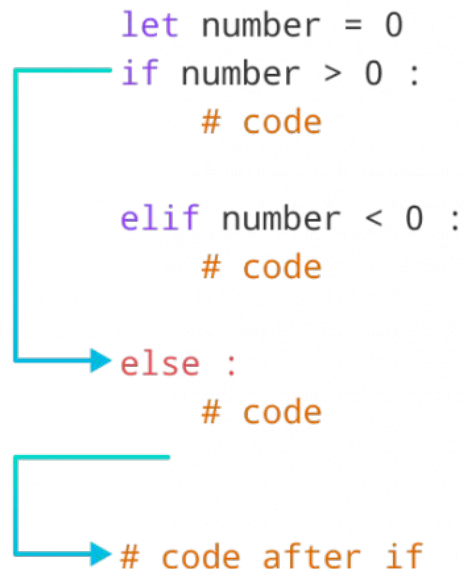
2nd Condition is True

```
let number = -5
if number > 0 :
    # code
elif number < 0 :
    # code
else :
    # code
# code after if
```



All Conditions are False

```
let number = 0
if number > 0 :
    # code
elif number < 0 :
    # code
else :
    # code
# code after if
```



Loops!

LOOP TYPE	DESCRIPTION
'for' loop	Used to iterate over a sequence of elements, such as a list or string
'while' loop	Used to repeat a block of code while a certain condition is true

```
numbers = [1, 2, 3, 4, 5]
```

```
for number in numbers:  
    print(number)
```

FOR
LOOP

```
1  
2  
3  
4  
5
```

INPUT

OUTPUT

```
count = 5
```

```
while count > 0:  
    print(count)  
    count -= 1
```

WHILE
LOOP

```
5  
4  
3  
2  
1
```

Functions are also just like what you've seen in a math class, but fancier

FUNCTION TYPE	DESCRIPTION
Built-in functions	Functions that are built in python and can be used without needing to define them. Examples - 'print()', 'len()' and 'range()'
User-defined functions	Functions that you create yourself to perform a specific task. They can be reused multiple times throughout your code.

```
print("Hello, world!")
```



Hello, world!

```
def add_numbers(a, b):  
    sum = a + b  
    return sum  
  
result = add_numbers(2, 3)  
print(result)
```



5

Packages make our lives easy

SciPy documentation

Date: January 20, 2024 Version: 1.12.0

Download documentation: <https://docs.scipy.org/doc/>

Useful links: [Install](#) | [Source Repository](#) | [Issues & Ideas](#) | [Q&A Support](#) | [Mailing List](#)

SciPy (pronounced "Sigh Pie") is an open-source software for mathematics, science, and engineering.

```
[ ]: import scipy
```

NumPy



The fundamental package for scientific computing with Python

LATEST RELEASE: NUMPY 1.26. VIEW ALL RELEASES

```
[ ]: import numpy as np
```

matplotlib

[Plot types](#) [User guide](#) [Tutorials](#) [Examples](#) [Reference](#) [Contribute](#) [Releases](#)



```
[ ]: import matplotlib.pyplot as plt
```

Matplotlib: Visualization with Python

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. Matplotlib makes easy things easy and hard things possible.